DSA Lab 1
Part A

1. Build data structure vector using dynamic arrays.
Functions to be implemented today:

`size()`: Returns the number of elements in the vector.

`capacity()`: Returns the size of the storage space currently allocated for the vector, expressed in terms of elements.

`empty()`: Checks whether the vector is empty (i.e., whether its size is 0).

`push_back(const int& value)`: Adds an element to the end of the vector.

`pop_back()`: Removes the last element in the vector.

`clear()`: Removes all elements from the vector, leaving it with a size of 0.

`operator[]`: Provides access to the element at the specified position using the subscript operator.

print(): Prints all elements of the vector.

2. Build data structure stack using your vector.
Functions:

`push(const T& value)`: Adds an element to the top of the stack.

`pop()`: Removes the top element from the stack.

`top()`: Returns a copy of the top element in the stack.

`empty()`: Checks whether the stack is empty.

`size()`: Returns the number of elements in the stack.

2. Build data structure queue using your vector.
Functions:

`push(const T& value)`: Adds an element to the end of the queue.

`pop()`: Removes the front element from the queue.

`front()`: Returns a copy of the front element in the queue.

`back()`: Returns a copy of the last element in the queue.

`empty()`: Checks whether the queue is empty.

`size()`: Returns the number of elements in the queue.

Part B

STACK

1. Reverse the Elements of a String

Write a program that uses a stack to reverse the characters of a given string. The program should take a string as input and output the reversed version of the string.

2. Parenthesis Validation

Create a program that uses a stack to validate whether the parentheses in a given expression are correctly matched. The program should return true if the parentheses are balanced, and false otherwise.

3. Implement Stack Using Queues

Develop a program to implement the functionality of a stack using one or more queues. Your implementation should support standard stack operations like push, pop, and top.

QUEUE

1. Reverse the First k Elements of a Queue

Create a program that reverses the first k elements of a queue. The program should take a queue and an integer k as input, and output the modified queue with the first k elements reversed while keeping the rest of the queue unchanged.

2. Remove a Given Element from a Queue

Design a program that removes a specific element from a queue. The program should take a queue and an element as input, and output the queue with the specified element removed.

3. Implement Queue Using Stacks

Create a program to implement the functionality of a queue using two stacks. Your implementation should support standard queue operations like enqueue, dequeue, and front.
